

DSA Interview Roadmap

Hey chatgpt, Act as my personal coach. i need to crack one of the FANG companies. for that i need to be strong on DSA. but all ik is coding that's it. some easy level prblms ik. but that too not on optimed approach. i nneed to get locked in for my winter arc, to get prepared for interviews, where i need to know about DSA. ik little about basics, like stack, queue and linked list but not sure of other things likw DP , graph , hashing and all. prepare me an roadmap and help me to crack the interview.Based on my previous coding related doubts also u can guide me where i need to focus and on what i need to focus first.

Show moreShow less

Absolutely. And based on the way you've approached your previous coding doubts, I **would not start by throwing 300 LeetCode problems at you.**

Your main gap isn't "you can't code." Your gap is:

You can implement things, but you don't yet have a strong pattern-recognition + algorithmic problem-solving framework.

That is fixable.

You already have some exposure to Java, stacks, queues, linked lists, recursion, and you recently worked through **House Robber/DP**. The fact that you were questioning *why* a DP state should choose one path over another is actually exactly the kind of thinking we want to develop.

Your FANG Interview Winter Arc

I'm going to structure this as a **5-6 month journey**, assuming roughly **2-3 hours/day on weekdays + more on weekends.**

The goal isn't merely:

"I solved 200 LeetCode questions."

The goal is:

I see a problem → identify the pattern → formulate the approach → prove/understand it → code it → analyze complexity → handle edge cases.

The Roadmap

Think of your preparation as 6 levels.

```

LEVEL 0
Programming fundamentals
  ↓
LEVEL 1
Arrays + Strings + Hashing
  ↓
LEVEL 2
Two Pointers + Sliding Window + Stack + Binary Search
  
```

```

↓
LEVEL 3
Linked List + Trees + Heaps + Intervals
↓
LEVEL 4
Graphs + Backtracking + Greedy
↓
LEVEL 5
Dynamic Programming
↓
LEVEL 6
Interview Mode
↓
FAANG/FANG

```

But there's an important point:

Don't treat these as isolated data structures.

We're going to learn **patterns**.

First: Where I Think You Currently Are

Based on your previous questions, I'd roughly put you here:

Area	Current level
Java coding	 Good foundation
Basic data structures	 Some exposure
Arrays/Strings	
Hashing	
Two pointers	
Sliding window	
Binary search	
Linked List	
Stack/Queue	
Trees	
Heap/Priority Queue	
Graphs	
Recursion	
Backtracking	
Greedy	
DP	 → 
Complexity analysis	
Pattern recognition	
Optimizing brute force	

And **pattern recognition + optimization** should be your biggest focus.

Because you already said something very important in an earlier discussion:

"With brute force I can solve, but with DSA I can't."

That is actually a very common stage.

We're going to turn:

```
Brute Force
↓
Why is it slow?
↓
What work is repeated?
↓
Can I store something?
↓
Can I eliminate unnecessary work?
↓
Which DSA/pattern does that suggest?
↓
Optimized solution
```

into your natural thought process.

PHASE 1 — Foundation

Weeks 1-2

1. Big-O

Before going deep into DSA, you need to become comfortable with:

```
O(1)
O(log n)
O(n)
O(n log n)
O(n²)
O(2ⁿ)
O(n!)
```

Not just memorizing them.

You should be able to look at:

```
Java
for (int i = 0; i < n; i++) {
    ...
}
```

and immediately think:

```
O(n)
```

And:

```
Java
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        ...
    }
}
```

→ $O(n^2)$

We'll practice this heavily.

● PHASE 2 — Arrays + Strings + Hashing

Weeks 3-5

This is where I want you to start seriously.

Learn:

- Arrays
- Strings
- HashMap
- HashSet
- Frequency counting
- Prefix sum
- Sorting
- Two pointers
- Sliding window

These are **extremely important** because they appear everywhere.

Example

Suppose:

```
nums = [2,7,11,15]
target = 9
```

Your first thought might be:

```
Try every pair
```

That's:

```
 $O(n^2)$ 
```

But then we ask:

"What information do I need to know when I'm looking at `nums[i]`?"

For 2, I need:

```
9 - 2 = 7
```

If I can remember previously seen numbers:

```
HashMap
```

we get:

```
O(n)
```

That's the kind of transformation I want you to learn.

● PHASE 3 — Core Interview Patterns

Weeks 6–9

Now we'll attack the patterns that make LeetCode much easier.

Two Pointers

Learn problems like:

```
Two Sum II
Valid Palindrome
3Sum
Container With Most Water
Trapping Rain Water
```

The important question becomes:

"Can I solve this by moving two boundaries instead of checking every possibility?"

Sliding Window

This is HUGE.

You'll learn:

```
Fixed-size window
Variable-size window
Frequency-based window
```

Problems:

```
Maximum Average Subarray
Longest Substring Without Repeating Characters
Longest Repeating Character Replacement
Minimum Window Substring
Permutation in String
```

You'll start seeing:

```
left ————— right
```

instead of immediately writing nested loops.

● Binary Search

Don't learn binary search as merely:

"Find an element in sorted array."

That's too basic.

Learn the **binary search pattern**.

```
left
mid
right
```

and eventually:

```
Binary Search on Answer
```

Problems like:

```
Search in Rotated Sorted Array
Find Minimum in Rotated Sorted Array
Koko Eating Bananas
Capacity to Ship Packages
```

The last two are especially important because the array isn't necessarily what you're searching.

You're searching the **answer space**.

● PHASE 4 — Linked Lists + Trees + Heaps

Weeks 10–13

You already know a little linked list, so we'll strengthen it.

Linked List patterns

Learn:

```
Fast & Slow pointers
Reverse linked list
Merge lists
Cycle detection
Middle node
```

Important problems:

Reverse Linked List
 Linked List Cycle
 Middle of Linked List
 Merge Two Sorted Lists
 Remove Nth Node From End
 Reorder List

Trees

This will be a major milestone.

Learn:

Binary Tree
 BST
 DFS
 BFS
 Tree recursion
 Height
 Depth
 Diameter
 Lowest Common Ancestor

And understand:



Instead of thinking:

"This is a tree problem."

you should start thinking:

"Is this DFS?"

or

"Is this BFS?"

or

"Do I need information from both children?"

That's pattern recognition.

Heap / Priority Queue

Learn:

Min Heap
 Max Heap
 PriorityQueue in Java
 Top K

Kth Largest
K-way merge

Problems:

Kth Largest Element
Top K Frequent Elements
Merge K Sorted Lists
Find Median from Data Stream

● PHASE 5 — Graphs

Weeks 14-17

This is where many beginners get scared.

Don't.

Graphs become much easier once you understand the templates.

You'll learn:

Graph representation
Adjacency List
DFS
BFS
Visited array
Connected Components
Cycle Detection
Topological Sort
Union Find
Shortest Path

Then:

Dijkstra

and eventually:

MST
Kruskal
Prim

You don't need to memorize 50 graph algorithms.

You'll learn:

When do I use which one?

● PHASE 6 — Backtracking + Greedy

Weeks 18-20

Backtracking:

```
Decision
↓
Choose
↓
Explore
↓
Undo
```

You'll solve:

```
Subsets
Permutations
Combination Sum
N-Queens
Word Search
```

And you'll finally understand why recursion is so powerful.

● PHASE 7 — Dynamic Programming

Weeks 21-25

This is where we're going to spend **a LOT of time with you**.

Because you've already started House Robber, and I saw the exact kind of confusion you're having.

DP isn't:

"Memorize DP formulas."

It's:

```
What is my state?
↓
What decision am I making?
↓
What happens after that decision?
↓
What smaller problem remains?
↓
Can I reuse its answer?
```

For House Robber:

```
dp[i] = maximum money we can rob from houses 0...i
```

Then:

```
Don't rob i
↓
dp[i-1]

Rob i
↓
nums[i] + dp[i-2]
```

Therefore:

```
dp[i] = max(
    dp[i-1],
    nums[i] + dp[i-2]
)
```

This is exactly the kind of reasoning I want you to become comfortable with.

DP Roadmap

We'll go:

Level 1

```
House Robber
Climbing Stairs
Min Cost Climbing Stairs
```

Level 2

```
Coin Change
Partition Equal Subset Sum
Target Sum
Longest Increasing Subsequence
```

Level 3

```
Unique Paths
Grid problems
Longest Common Subsequence
Edit Distance
```

Level 4

```
Knapsack
Interval DP
Advanced DP
```

You don't need advanced DP immediately.

Your Weekly System

This is **more important than the roadmap**.

Don't do:

```
Monday → 10 problems
Tuesday → 10 problems
Wednesday → 10 problems
```

You'll forget most of them.

Instead:

Monday

Learn pattern

Example:

```
HashMap / Frequency Counting
```

Then solve 2 easy problems.

Tuesday

Solve:

```
2 easy  
1 medium
```

WITHOUT looking at the solution.

Wednesday

Take yesterday's medium.

Try again.

Then learn another pattern.

Thursday

```
2 medium
```

Friday

Revision day

No new topic.

Take 3 previously solved problems.

Try to solve them from scratch.

Saturday

🔥 Deep practice

```
3-4 problems
```

and one timed problem.

Sunday

Rest / light revision.

And maintain an **error notebook**.

Your DSA Error Notebook

This will be extremely valuable for you.

For every problem you fail, write:

```

Problem:
Two Sum

My approach:
Brute force  $O(n^2)$ 

Why it failed:
Too slow

Optimal pattern:
HashMap

Key insight:
For nums[i], search target - nums[i]

Complexity:
 $O(n)$  time
 $O(n)$  space

Mistake:
Didn't think about storing previous values.

```

After 100 problems, this becomes your personal DSA textbook.

The Most Important Rule For You

Whenever you see a new problem:

DON'T immediately look for the optimal solution.

First do this:

Step 1 — Understand

What exactly is being asked?

Step 2 — Brute force

Ask:

"If constraints didn't matter, how would I solve this?"

Step 3 — Complexity

Calculate:

```
Time = ?
Space = ?
```

Step 4 — Find the bottleneck

Ask:

"What part is making my solution slow?"

Step 5 — Look for a pattern

Ask:

```
Can HashMap help?
Can sorting help?
Can two pointers help?
Can sliding window help?
Can binary search help?
Can stack help?
Can heap help?
Can recursion help?
Can DP help?
Can graph traversal help?
```

Step 6 — Optimize

Then write the solution.

Step 7 — Explain

You should be able to explain it to me without looking.



One Thing I DON'T Want You To Do

Don't start doing:

```
Blind 75
NeetCode 150
LeetCode Top 100
```

just because everyone else does.

Those lists are useful **after your fundamentals are strong enough**.

Right now, if I give you:

"Solve this Medium."

and you don't know whether it's:

```
HashMap?
Sliding Window?
Binary Search?
DFS?
BFS?
DP?
Greedy?
```

you're going to struggle unnecessarily.
We're going to build that recognition first.

Your Target Problem Count

Don't obsess over numbers, but a good target is roughly:

```
Easy      ~60
Medium    ~120
Hard      ~20
-----
Total     ~200
```

But **200 understood problems > 500 memorized solutions.**

For FANG-level interviews, I'd rather you know:

150–200 problems deeply

than 500 superficially.

Your Daily 2–3 Hour Routine

If you have ~2.5 hours:

```
20 min
Concept learning

60 min
Problem solving

20 min
Study optimal solution / hints

30 min
Re-code without looking

20 min
Review old problems
```

And importantly:

Don't spend 2 hours staring at one problem.

Use this rule:

```
0–20 min → struggle yourself
20–30 min → hints
30+ min → study solution
```

Then close the solution and **implement it yourself.**

That is learning.

And We're Going To Use Java

Since Java is already your language, we're sticking with it.

I want you to become very comfortable with:

Java

*HashMap
HashSet
ArrayList
ArrayDeque
PriorityQueue
Arrays
Collections
StringBuilder
Comparator*

and eventually:

Java

record / class

for graph/tree structures where appropriate.

You shouldn't lose an interview because you forgot how **PriorityQueue** works.

Your Final Interview Phase

After the DSA foundation:

Round 1 — Pattern recognition

I'll give you a random problem.

You tell me:

*What pattern?
Why?*

without coding.

Round 2 — Coding

You get:

30-40 minutes

Solve it completely.

Round 3 — Follow-up

I'll ask:

Can you optimize this?

or:

What if the input is huge?

or:

What if duplicates are allowed?

This is where interview thinking develops.

Round 4 — Mock interviews

We'll simulate:

```

Interviewer
↓
You
↓
Clarifying questions
↓
Brute force
↓
Optimization
↓
Code
↓
Test
↓
Complexity
↓
Follow-up

```

And Here's How I Want To Coach You

Don't think of me as:

"Give me the solution."

Use me as:

"Train my brain to find the solution."

For every problem you bring me, I'll follow this structure:

```

1. Understand the problem
↓
2. Build intuition
↓
3. Brute force
↓
4. Why brute force is bad
↓
5. Identify DSA pattern
↓
6. Derive optimal approach
↓
7. Dry run
↓
8. Java implementation
↓
9. Time & space complexity
↓

```

```
10. Edge cases
    ↓
11. Interview-style follow-up
```

And **I won't immediately dump the final code on you** unless you ask for it. I'll make you think first.

Your Starting Point

Don't start Graphs.

Don't start DP.

Don't start Hard problems.

Don't even start Trees yet.

Your first journey should be:

```
BIG-O
↓
Arrays
↓
Strings
↓
HashMap / HashSet
↓
Sorting
↓
Two Pointers
↓
Sliding Window
↓
Binary Search
↓
Stack / Queue
```

Then:

```
Linked List
↓
Trees
↓
Heap
↓
Graphs
↓
Backtracking
↓
Greedy
↓
DP
```

That's the order I'd recommend **specifically for where you are right now**.

And one more thing: **don't worry about being "late."** You're at the exact stage where structured practice can make a huge difference. Your ability to code is already the base; now we need to build the algorithmic thinking on top of it.

Your Winter Arc mission:

Not "solve LeetCode."

Become the person who can look at an unfamiliar problem and systematically figure out how to solve it.

If you're locked in, **Day 1 starts with Big-O + Arrays + the first pattern: Hashing.** We can treat this like an actual coaching program and progress topic-by-topic rather than randomly jumping between LeetCode questions.